FOTA 包制作流程介绍

目前 ASR 项目中升级主要有三种：DFOTA(非小系统差分)、FullFOTA(全量升级)、MiniFOTA（小系统升级）。

其中 DFOTA 升级方案制作差分包时又分为两种：FBF 差分以及 adiff 差分，他们两者的区别主要为压缩后差分还是压缩前差分，压缩后差分会比压缩前差分得到的包要大，所以 FBF 差分包占用空间更大；但是它也有其优势，就是 FBF 默认是每 64K 分包差分，那么备份分区 fota_pkg 也就只需要这么大，不像 adiff 差分，是对整个内存小分区进行差分，那么备份时需要的空间就必须要大于包中任意一小分区的大小（1M+），而这个分区平时不使用，也造成了空间的浪费。

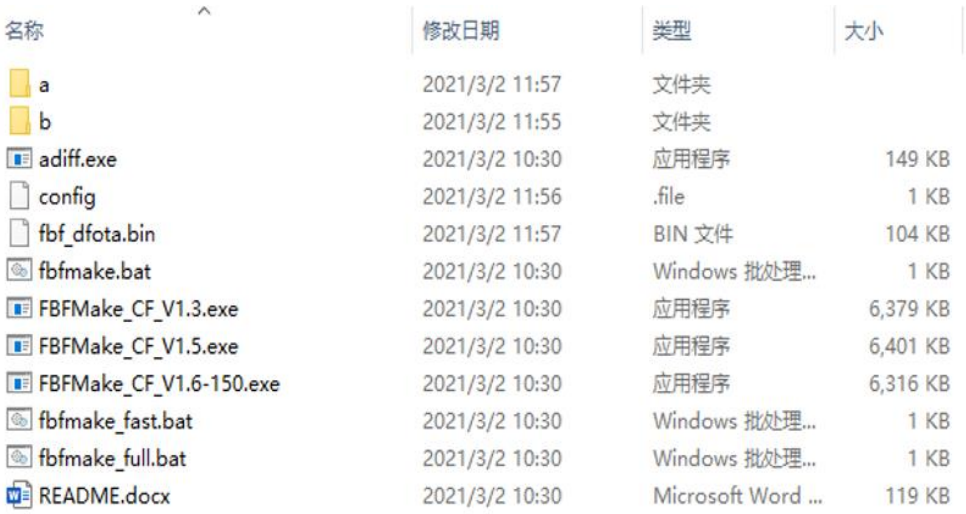
MiniFOTA 表述上是小系统差分升级，但实际上做差分的只是小系统代码部分，当小系统差分升级完成后，大系统升级实际上是全量升级（不论是 cp 还是 app 或其他镜像），小系统升级大系统时，下载的大系统全量包会保存在 ram 中（由于处于小系统中，ram 足够大，即使下载远快于升级，也能存下），边下边升，这也就解释了后面制作差分包时升级其他分区为什么直接跟-a+分区名+目标镜像，而不带 old 镜像，且此升级方式没有备份分区 FOTA_pkg，所以尽管在 1606 上 8M 项目支持非小系统差分升级，但由于没有备份分区，所以无法避免掉电会导致模块变砖问题。

三种方式对比如下：

功能	DFOTA	minsys	FULL FOTA
差分包存储位置&数量	CUSTOMER_FS (要求空间剩余>2M)	客户无需关注，**2个包** 不占用FS空间	无差分包
全量包存储位置&数量	无	客户无需关注	CUSTOMER_FS (要求空间剩余=APP ROM)
差分包/全量包下载方式	1、客户自行存储到文件系统 2、移远提供一键下载升级【需要开启移远内置的HTTP/FTP功能】	移远提供一键下载升级【HTTP/FTP】	1、客户自行存储到文件系统 2、移远提供一键下载升级【需要开启移远内置的HTTP/FTP功能】
差分包/全量包存储位置	FS（要求客户在下载时>2M）	不占用FS空间	全包大小决定占用空间
是否支持LCD升级进度显示	支持	不支持	不支持
是否支持掉电续升	支持	支持	支持
APP	支持	支持	支持
KERNEL	支持	支持	不支持
是否需要备份	是，fbf方式需要64K，adiff方式以最大分区大小计算，至少1.5M+，使用fota_pkg分区	无	无
重启次数	2次	4次	2次
	要求FS空间有足够空间	重启次数多	速度快

制作方法：

升级包制作工具的目录如下图所示：



名称	修改日期	类型	大小
a	2021/3/2 11:57	文件夹	
b	2021/3/2 11:55	文件夹	
adiff.exe	2021/3/2 10:30	应用程序	149 KB
config	2021/3/2 11:56	.file	1 KB
fbf_dfota.bin	2021/3/2 11:57	BIN 文件	104 KB
fbfmake.bat	2021/3/2 10:30	Windows 批处理...	1 KB
FBFMake_CF_V1.3.exe	2021/3/2 10:30	应用程序	6,379 KB
FBFMake_CF_V1.5.exe	2021/3/2 10:30	应用程序	6,401 KB
FBFMake_CF_V1.6-150.exe	2021/3/2 10:30	应用程序	6,316 KB
fbfmake_fast.bat	2021/3/2 10:30	Windows 批处理...	1 KB
fbfmake_full.bat	2021/3/2 10:30	Windows 批处理...	1 KB
README.docx	2021/3/2 10:30	Microsoft Word ...	119 KB

图：升级包制作工具目录

升级包制作工具的主要目录文件介绍如下，其他目录无需关注：

- 文件夹 *a*（after 的简称）：用于存储新版本固件包中的 *system.img* 文件；
- 文件夹 *b*（before 的简称）：用于存储旧版本固件包中的 *system.img* 文件；
- *config*：用于存储单个分区或者多个分区配置文件；
- *fbfmake_fast.bat*：用于制作旧基线差分升级包；
- *fbfmake_full.bat*：用于制作整包升级包。

通过如下方法完成升级包制作后，将升级包上传至服务器上，在模块上启动 FOTA 功能，即可进行升级包的下载和升级。

1. FullFOTA 包制作流程：

在 SDK 中首先找到 ql-sdk\ql-bootloader\updater\tool 目录，此目录下是制作所有类型 fota 包均需使用的脚本文件。

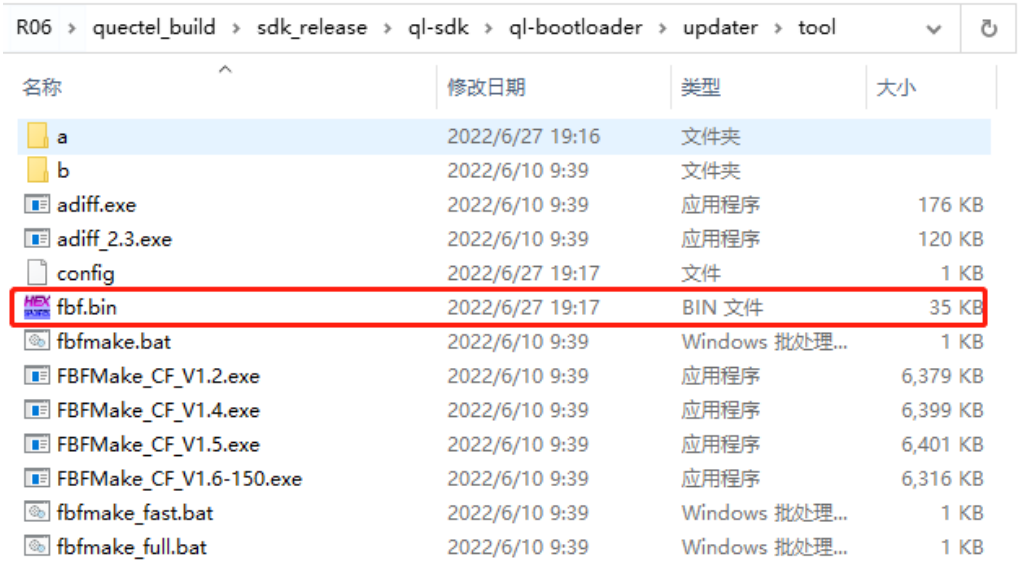
步骤一：按需修改此目录下的 config 文件，以下以 customer_app 分区为例：

```
[Image_List]
Number_of_Images = 1 需要制作的分区个数
1_Image_Enable = 1 使能此分区是否参与升级包的制作
1_Image_Image_ID = 0x30
1_Image_Path = customer_app.bin  分区名
1_Image_Flash_Entry_Address = 0x005D7000 分区偏移地址
1_Image_ID_Name = 1
```

图：修改配置文件-制作全量包

步骤二：将应用侧编译生成的 app.bin 重命名为 config 中配置的文件名，此处即 customer_app.bin，并将其拷贝到 tool 目录下的 a 文件夹中。

步骤三：左键双击执行 tool 目录下的 fbfmake_full.bat 脚本文件，执行成功后在升级包制作工具根目录下生成全量升级包 fbf.bin。如下图所示，该升级包比原始镜像增大 8 KB，增大的 8 KB 数据中包含了校验信息和分区定位信息。



图：全量包

至此，fota 的全量包也就制作完成，不过也是由于全量包是将整个镜像加上校验和分区定位信息组成，所以导致升级包会比原包跟大，而如果想用此来升级 cp、dsp 等较大的镜像，则需要的 ram 必须足够，否则无法保存下载的升级包，**也就不推荐采用全量升级来升级 cp 等较大镜像**。

2. fbf 差分包制作流程

同样也是找到 SDK 中的 ql-sdk\ql-bootloader\updater\tool 目录，然后执行以下操作:

步骤一：修改 tool 目录下的 config 文件，此流程同全量包中步骤一是一样的，不过正是由于差分包相对较小，所以 config 中配置的镜像可以更多，以 system 和 customer_app 分区进行差分升级为例，需配置信息如下:

```
[Image_List]
Number_of_Images = 2
1_Image_Enable = 1
1_Image_Image_ID = 0x30
1_Image_Path = system.img
1_Image_Flash_Entry_Address = 0x00016000
1_Image_ID_Name = 1
2_Image_Enable = 1
2_Image_Image_ID = 0x31
2_Image_Path = customer_app.bin
2_Image_Flash_Entry_Address = 0x005D7000
2_Image_ID_Name = 2
```

图：修改配置文件-制作差分包

步骤二：将新版本固件包中的 system.img 和 customer_app.bin 文件拷贝至 a 目录下，将旧版本固件 包中的 system.img 和 customer_app.bin 文件拷贝至 b 目录下。。

步骤三：左键双击执行 fbfmake_fast.bat 脚本文件。执行脚本文件后开始制作差分升级包，脚本执行完 成后会在升级包制作工具根目录下生成差分升级包。

命令详解:
fbfmake_fast.bat 主要执行的命令为:
FBFMake_CF_V1.6-150.exe -f config -d 0x10000 -a a -b b -o fbf_dfota.bin -q
-f：制作差分包需要的配置文件，此处即 config 文件。
-d：代表差分升级使用的切割块大小，一般为 0x10000, 64K。
-a：（after）设置升级后的 image 文件路径，比如 a 文件夹下会放入升级后版本的 system.img。
-b：（before）设置升级前的 image 文件路径，比如 b 文件夹下会放入升级前版本的 system.img。
-o：代表输出的差分文件，为 fbf_dfota.bin
-q：可选参数，如果选择该参数，差分将选择 fast 模式，制作差分包的时间将大大缩小，但是该模式可能会导致差分包比非 fast 模式要大，在大部分情况下 fast 模式和非 fast 模式输出差分包大小是基本相同。

由于使用 fbf 差分制作的包较大，所以推荐使用其后续优化版本 adiff 差分方法制作。

3. adiff 差分制作流程

步骤一：将新旧固件包中的 system.img 拷贝至升级包制作工具根目录下并分别命名为 system_new.img 和 system_old.img；并将新旧固件包中的 customer_app.bin 文件拷贝至升级包制作工具根目录下并分别命名为 app_new.bin 和 app_old.bin（镜像名不是必须同上述保持一致，但需要与命令中名称一致）。

步骤二：打开命令行并输入执行如下命令开始制作差分包：

```
adiff.exe -l fp system_old.img system_new.img fbf_dfota.bin
-a1 customer_app app_old.bin app_new.bin
```

执行完成后会在 tool 目录下生成名为 fbf_dfota.bin 的文件，该文件即为制作完成的差分升级包。

由于此方式是压缩后差分，所以得到的包相对较小，推荐使用。

4. 小系统差分制作流程

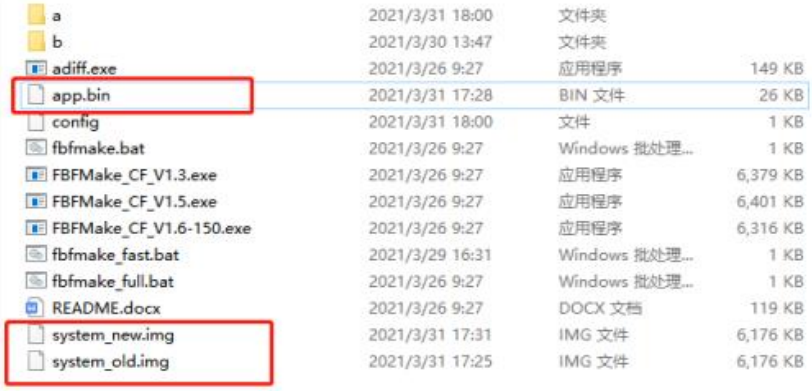
步骤一：拷贝相关文件至升级包制作工具目录。

1. 若仅对 system 分区进行差分升级，将新旧固件包中的 system.img 文件拷贝至升级包制作工具根目录（需与 adiff.exe 位于同级目录），并分别命名为 system_new.img 和 system_old.img。



图：放置 system.img 文件

2. 如需同时对 system 和其他分区进行差分升级，还需额外将新固件包中的其他分区镜像文 件拷贝至升级包制作工具根目录（需与 adiff.exe 位于同级目录）。以差分升级 system 分 区和 customer_app 分区为例，还需将 customer_app.bin 文件拷贝至升级包制作工具根 目录并重命名该文件为 app.bin。



图：放置 system.img 和 app.bin 文件

步骤二：在 tool 目录下打开命令行，执行如下命令：

```
adiff.exe system_old.img system_new.img old-new.bin
<Options>
[-a1 <app1_partition_name> <app1_file_name>
[-a2 <app2_partition_name> <app2_file_name>] ]
```

前三个参数是必选项，-a1 是可选项，-a2 是可选项。
system_old.img 参数是板子中已经烧写的 system 文件，如 1601 升级包 system.img 重命名为 system_old.img;

system_new.img 参数是要升级到的 system 文件，如 1601 升级包 system.img 重命名为 system_new.img;

system_patch.bin 参数是生成的升级文件名称；

-a1 代表当系统中存在和 system 同级的客户分区 app1，如下图所示，分区 app1_partition_name 名称假设为 app1，当要升级这个分区时可以使用这个参数，并将要升级的 app1_file_name 文件放在 **adiff.exe** 同一路径下，app1_file_name 文件在工具制作时会被放入 **system_patch.bin_2** 中，在第二阶段升级；
（app1 分区名称客户可以在 aboot 工具下修改，这种分区主要是给客户增加一些特殊的分区，然后也能够升级，使整个最小系统升级更灵活）
-a1 也可以指定为 updater 分区，这样在第二阶段升级就可以升级 updater，命令如下：
adiff.exe system_old.img system_new.img system_patch.bin –a1 updater updater.bin

app1_partition_name 代表分区的分名称；
app1_file_name 代表要升级的分区文件；

-a2 同上 **a1**，最小系统升级最多支持同时升级两个 app 分区；（注意 adiff 只能最多添加两个 app 分区用于升级）

小系统升级需要注意的是：调用 api 之后，会先重启进入小系统，然后在小系统中下载新的小系统代码，当下载结束后，继续重启，再升级小系统代码到目标版本的小系统，升级结束后重启，再进入新的小系统中，此时会边下载大系统代码边直接升级大系统代码，升级完成后再次重启后，升级也就结束了。所以 minifota 总共会重启 4 次才会升级成功。

5. 常用函数说明

int ql_fota_firmware_download(int profile_idx, qlFotaDwnldMod mode, char * address, char * username, char * password, qlFotaProgressCB_t cb);

该函数用于启动升级包下载

参数
profile_idx: [In] 数据拨号的数据通道号。

mode： [In] 下载协议参考枚举 qlFotaDwnldMod，也就是选择 FTP 还是 HTTP。
address： [In] 升级包存储至服务器上的地址。
username： [In] 服务器的登录名。若无，NULL。
password： [In] 服务器的登录密码。若无，NULL。
cb： [In] 下载进度状态的回调函数，用于表示下载进度和返回错误码。

返回值 0 函数执行成功 -1 函数执行失败

错误码参考：

参数	描述
(0) *FOTA_SUCCESS*	下载成功
(100) *FOTA_COMPLETED*	下载完成
(1001) *FOTA_DOMAIN_NOT_EXIST*	下载地址不存在
(1002) *FOTA_DOMAIN_TIMEOUT*	下载地址访问超时
(1003) *FOTA_DOMAIN_UNKNOWN*	下载地址格式错误
(1004) *FOTA_SERVER_CONN_FAIL*	服务器连接失败
(1005) *FOTA_AUTH_FAILED*	授权失败
(1006) *FOTA_FILE_NOT_EXIST*	文件不存在
(1007) *FOTA_FILE_SIZE_INVALID*	文件大小不可用
(1008) *FOTA_FILE_GET_ERR*	文件获取错误
(1009) *FOTA_FILE_CHECK_ERR*	升级包校验错误
(1010) *FOTA_INTERNAL_ERR*	内部错误
(1011) *FOTA_NOT_INPROGRESS*	下载中断
(1012) *FOTA_NO_MEMORY*	内存不足
(1013) *FOTA_FILE_SIZE_TOO_LARGE*	文件太大
(1014) *FOTA_PARAM_SIZE_INVALID*	无效参数

如果需要使用 FTPS 或 HTTPS 协议，则需要使用 void ql_fota_ssl_config(SSLConfig * sslCfg)接口配置，此接口的参数为 SSL 配置结构体。一般使用默认值，除域名地址和端口号外，其他无需修改。

ql_fota_firmware_download 函数集下载与启动升级为一体，只要设置好下载方式及地址等等信息，可自动完成后续升级工作，使用起来较为方便，但是不排除有客户需要自己单独下载差分包到模组，然后只是需要我们模组侧启动升级的要求，那么就可以使用以下这一组接口：

qlFotaImgProcCtxPtr ql_fota_init(void);

int ql_fota_image_write(qlFotaImgProcCtxPtr context, void * data, int len);

int ql_fota_image_read(qlFotaImgProcCtxPtr context, void * data, int len);

int ql_fota_image_flush(qlFotaImgProcCtxPtr context);

int ql_fota_image_verify(qlFotaImgProcCtxPtr context);

int ql_fota_image_verify_without_setflag(qlFotaImgProcCtxPtr context, int *fileSize, int *pIsDfota);

int ql_fota_set_update_flag(int file_size, int is_dfota);

void ql_fota_deinit(qlFotaImgProcCtxPtr context);

这组接口的参数可以参考 fota 相关文档查阅，这里主要说明一下每个函数的作用：

ql_fota_init 函数用来创建写入升级包数据的上下文结构体，并返回句柄，后续使用均是使用此句柄完成；

初始化后，我们可以通过 ql_fota_image_write 函数将数据写到 flash 上（比如 1603 上是写到文件系统 customer_fs.bin 中，实际此处是先将数据刷到准备好的一段 cache 缓存中，然后再将缓存中的数据全都写到文件系统的"U:/FotaFile.bin"文件中，实际上只要文件系统中有此同名文件即可，可以是客户预置或者使用文件系统接口写均可）；

```
while (len && context->flash_cache) {
    if ((context->flash_cache_index + len) >= FLASH_BLOCK) {
        int space = FLASH_BLOCK - context->flash_cache_index;
        memcpy(context->flash_cache + context->flash_cache_index, data, space);
        data += space;
        len -= space;
        context->flash_cache_index = 0;
#ifdef  FOTA_FS_DOWNLOAD
        if(context->fp == NULL)
        {
            if(offset==0)
            {   context->fp = ql_fopen(FOTA_FILE_NAME, "w+");
                fota_printf("%s offset:%d to open new fota file",__FUNCTION__,offset);
            }
            else
            {
                context->fp = ql_fopen(FOTA_FILE_NAME, "a+");
                fota_printf("%s append offset:%d\n",__FUNCTION__,offset);
            }
            if(context->fp == NULL)
            {
                fota_printf("%s fail to create fota file",__FUNCTION__);
                return FOTA_FILE_SAVE_FAIL;
            }
        }
        do
        {
            ret = ql_fwrite(context->flash_cache + data_written_len, data_left_len, 1, context->fp);
            ql_fsync(context->fp);
            data_written_len += ret;
            data_left_len -= ret;
            fota_printf("%s ql_fwrite() ret(%d) data_written_len = %08x processed_cnt = %08x\r\n",__FUNCT
        } while(data_written_len != FLASH_BLOCK && ret > 0);

        if(ret < 0)
```

当所有数据都写完成后，不能排除还有部分缓存没有及时写入，那么 cache 中就还有部分残余数据，我们就可以通过 ql_fota_image_flush 接口将 cache 中的剩余缓存数据刷到 flash 中，确保升级包的完整；

此时如果仅仅是想检验一下下载的包正不正确而不需要重启直接进升级流程，那么就可以使用 ql_fota_image_verify_without_setflag 接口，此接口会校验下载的升级包的头部信息以及包中所有分区的 crc 是否正确，如果正确，则会通过参数*fileSize 和*pIsDfota 返回升级包大小以及 fota 类型（0 是全量，1 是差分）；后续如果想要开始升级那就可以通过 ql_fota_set_update_flag 接口进行设置，传入的参数刚好是上一步返回的值。

而如果客户是想直接校验后重启就直接进入升级，则可以使用 ql_fota_image_verify 接口，此接口也会先校验 fota 包中的头部信息以及 crc，另外他还会配置 fota 参数区的参数并写入 fota 参数区（参数包括升级标志 upgrade_flag、升级方式 Upgrade_Method、是否

备份、fota_pkg 分区的偏移地址、fota 参数区的 crc 值等等），写入 fota 参数的主、备分区后会再次对主、备分区做 crc 校验，满足后返回；

至此，fota 相关设置完成，后续只需要重启模组即可进入升级流程。

关于此流程，可以参考 example_file_fota.c 中的使用，此测试用例是通过 main uart 串口来发送 fota 包，发送结束后主动重启进入升级。

```
    ql_uart_write(QL_MAIN_UART_PORT, data, sizeof(data));
    ql_rtos_flag_wait(fota_uart_flag, QL_FOTA_UART_READ, QL_FLAG_OR, &event, QL_WAIT_FOREVER);

    ctx = ql_fota_init();
    if(!ctx)
    {
        printf("*** fota init fail ***");
        return;
    }

    count = 0;
    while((ret >= 0) && (count < 15)){
        ret = ql_uart_read(QL_MAIN_UART_PORT, read_buf, sizeof(read_buf));
        if(ret <= 0){
            count++;
            ql_rtos_task_sleep_ms(10);
            continue;
        }else{
            count = 0;
            file_len += ret;
            res = ql_fota_image_write(ctx, (void *)read_buf, ret);
            if(res < 0){
                printf("ql_fwrite error:%d \n",res);
            }
        }
    }

    if(count >= 15){
        printf("Transfer end:%d ,file_len:%d\n",ret, file_len);
    }
    ql_uart_close(QL_MAIN_UART_PORT);

    ret = ql_fota_image_flush(ctx);
    if(ret)
    {
        printf("*** fota image flush fail ***");
        return;
    }
    printf("ql_fota_image_flush end\n");
    ret = ql_fota_image_verify(ctx);
    if(ret)
    {
        printf("*** fota image verify fail ***");
        return;
    }
    printf("ql_fota_image_verify success! \n");

    ql_fota_set_update_flag(file_len, 1);

    printf("ready to power reset!!\n");
    ql_power_reset();
} « end quec_fota_uart_test »
```

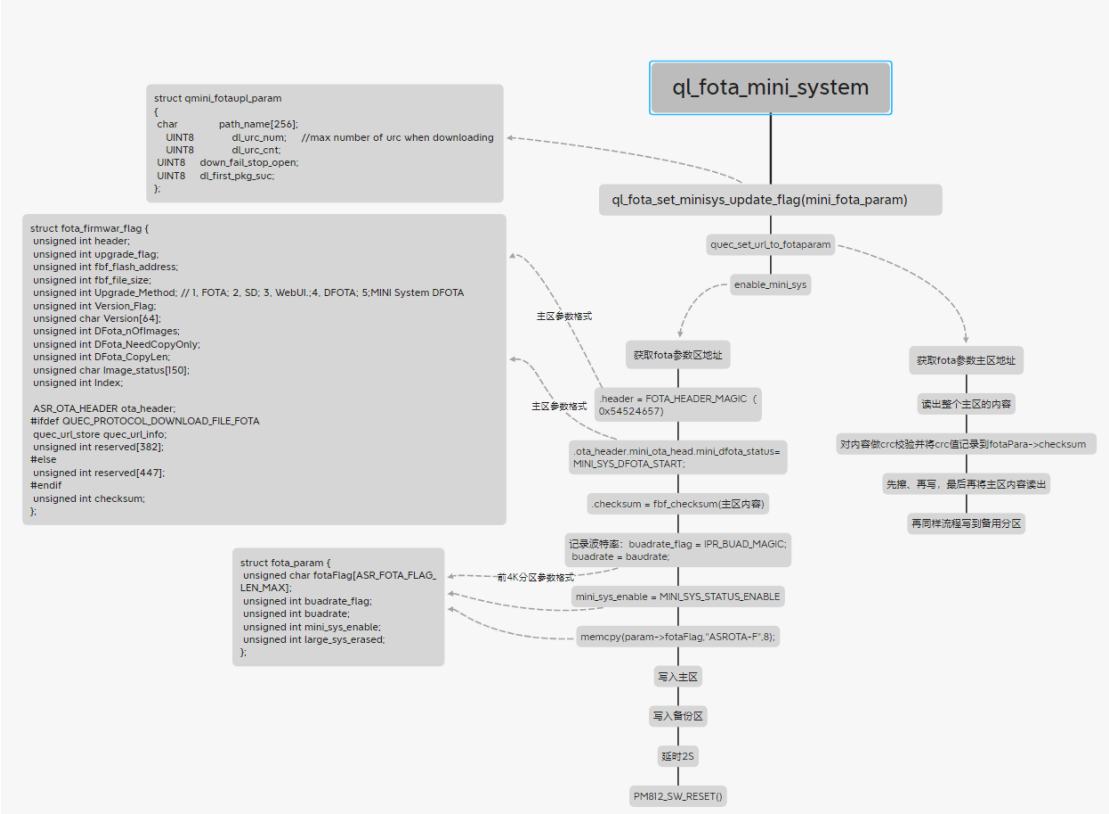
对于 minifota 功能，使用的 api 为：
int ql_fota_mini_system(int profile_idx, qlFotaDwnldMod mode, char * host, char * host2, char * username, char * password);

此接口同 ql_fota_firmware_download 接口类似，都是将下载方式、url 等信息填入后，模块会自动下载后进入升级程序，升级的 fota 参数自动写入。

注意：小系统接口中第二个参数 mode 不重要，主要为解析参数 host1/2 即 url 中的协议名称，如：

```
#define QL_FOTA_USE_URL "http://220.180.239.212:8300/Eve/system_patch.mini_1"
//#define QL_FOTA_USE_URL "https://112.31.84.164:8301/august/system_patch.mini_1"
// #define QL_FOTA_USE_URL "ftp://quec:quec@47.110.10.41:21/cara/system_patch.mini_1"
```

接口流程如下：



注意：如果在 8M 的项目中使用 DFOTA 方式进行升级，除了无 fota_pkg 造成掉电变砖的风险外，也需要将 updater.bin 跟换为 updater 目录编译出的 updater_no_backup.bin，这样才能正常升级，否则会卡住无法使用。